Deep Learning Techniques Laboratory

Ex: 7

Comparison of RNN, LSTM and GRU

Working:

This work focuses on the comparative analysis of Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) models for multivariate time-series forecasting using the Daily Delhi Climate dataset. The dataset contains 1,462 daily observations with four climate-related variables: mean temperature (`meantemp`), humidity, wind speed, and mean pressure, along with the corresponding date. The date attribute is converted into a datetime format and used as the temporal index. Exploratory data analysis is initially performed to understand the temporal behavior and distribution of the climate variables. Based on the observed time-series patterns, `meantemp` is selected as the target variable, while humidity, wind speed, and mean pressure are considered input features.

The dataset is divided chronologically into training and testing sets using an 80:20 split. Separate Min-Max scalers are used for the input features and the target variable. The scaled multivariate observations are then transformed into sequential input samples using a sliding-window approach. A window size of 30 days is selected, where the climate observations from 30 consecutive days are provided as input to predict the mean temperature of the following day. Consequently, each input sequence has a dimensionality of 30 time steps × 3 input features. Three deep learning architectures—RNN, LSTM, and GRU—are developed using comparable network structures to enable a fair performance comparison. The models are evaluated using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and execution time. The prediction performance of RNN, LSTM, and GRU is then compared to identify the model that provides the most effective forecasting performance for the given multivariate climate time-series data.

The experimental settings used are illustrated in Table 1.

Table 1. Training Configurations

Parameter	**Value**
Epochs	5
Batch size	64
Optimizer	Adam
Loss Function	Categorical Crossentropy
Validation split	20%
Embedding Dimension	256
RNN Units	256

RNN (SimpleRNN)

Purpose: Multivariate time-series forecasting

Input: Sequential climate data (30 days × 3 features)

Architecture: Three recurrent layers with 32, 64, and 128 units

Memory: Maintains information through the recurrent hidden state

Dropout: 0.2 after each recurrent layer

Output: Predicted next-day mean temperature.

Model Summary:

RNN Model:

Model: "sequential_10"

Layer (type)	Output Shape	Param #
simple_rnn_26 (SimpleRNN)	(None, 30, 32)	1,152
dropout_20 (Dropout)	(None, 30, 32)	0
simple_rnn_27 (SimpleRNN)	(None, 30, 64)	6,208
dropout_21 (Dropout)	(None, 30, 64)	0
simple_rnn_28 (SimpleRNN)	(None, 128)	24,704
dropout_22 (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 1)	129

Total params: 32,193 (125.75 KB)

Trainable params: 32,193 (125.75 KB)

Non-trainable params: 0 (0.00 B)

LSTM (Long Short-Term Memory)

Purpose: Multivariate time-series forecasting

Input: Sequential climate data (30 days × 3 features)

Architecture: Three LSTM layers with 32, 64, and 128 units

Memory: Uses memory cells and gating mechanisms to retain relevant temporal information

Dropout: 0.2 after each recurrent layer

Output: Predicted next-day mean temperature.

Model Summary:

```
LSTM Model:

Model: "sequential_13"
```

Layer (type)	Output Shape	Param #
lstm_9 (LSTM)	(None, 30, 32)	4,608
dropout_29 (Dropout)	(None, 30, 32)	0
lstm_10 (LSTM)	(None, 30, 64)	24,832
dropout_30 (Dropout)	(None, 30, 64)	0
lstm_11 (LSTM)	(None, 128)	98,816
dropout_31 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 1)	129

```
Total params: 128,385 (501.50 KB)

Trainable params: 128,385 (501.50 KB)

Non-trainable params: 0 (0.00 B)
```

GRU (Gated Recurrent Unit)

Purpose: Multivariate time-series forecasting

Input: Sequential climate data (30 days × 3 features)

Architecture: Three GRU layers with 32, 64, and 128 units

Memory: Uses update and reset gates to control the flow of temporal information

Dropout: 0.2 after each recurrent layer

Output: Predicted next-day mean temperature

Model Summary:

```
GRU Model:
```

Model: "sequential_14"

Layer (type)	Output Shape	Param #
gru_3 (GRU)	(None, 30, 32)	3,552
dropout_32 (Dropout)	(None, 30, 32)	0
gru_4 (GRU)	(None, 30, 64)	18,816
dropout_33 (Dropout)	(None, 30, 64)	0
gru_5 (GRU)	(None, 128)	74,496
dropout_34 (Dropout)	(None, 128)	0
dense_14 (Dense)	(None, 1)	129

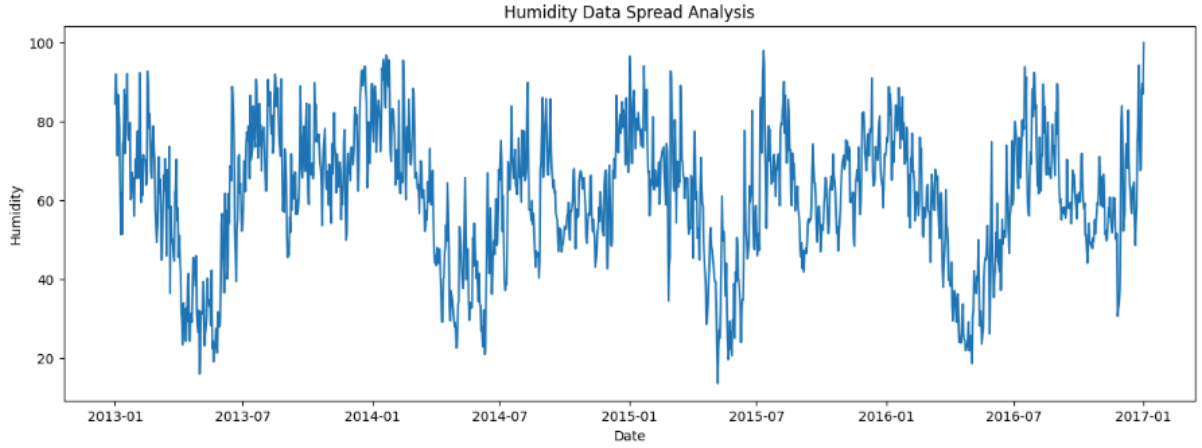
Total params: 96,993 (378.88 KB)

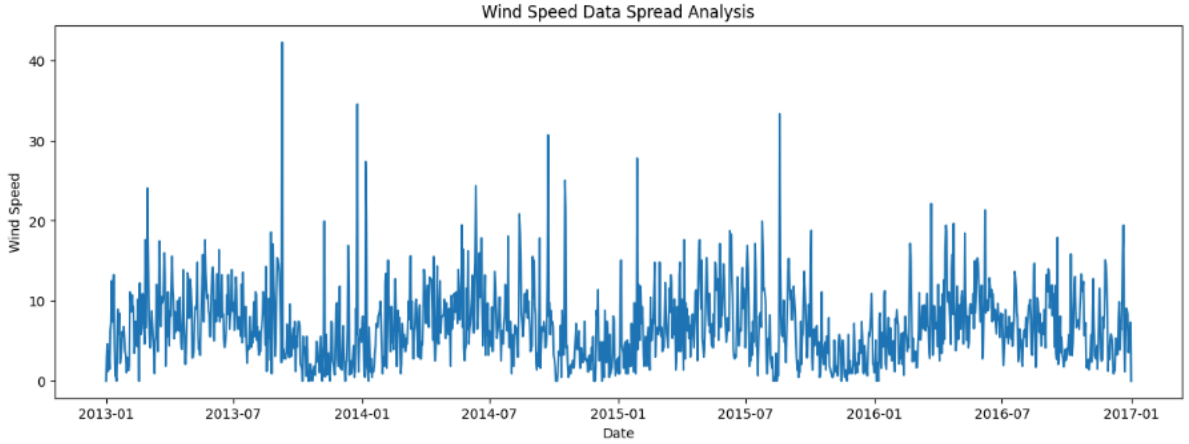
Trainable params: 96,993 (378.88 KB)

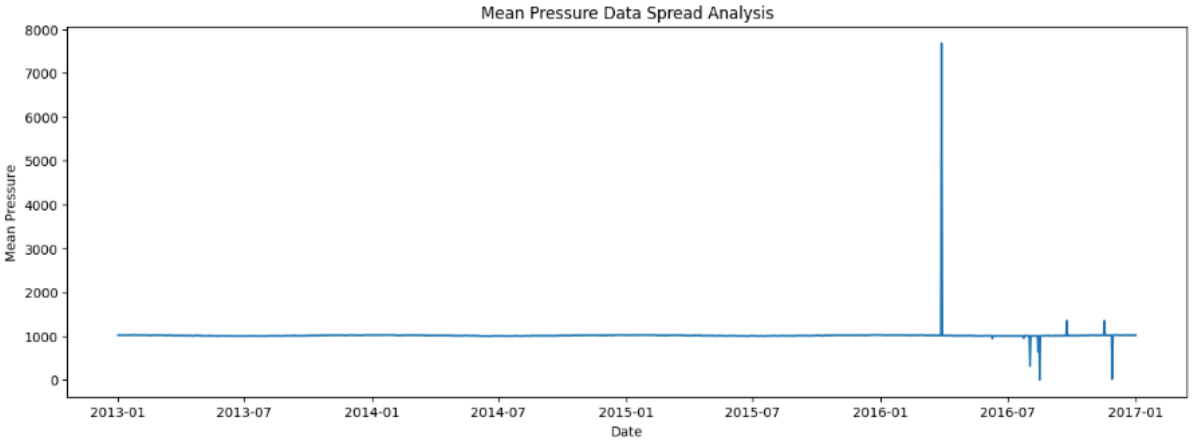
Non-trainable params: 0 (0.00 B)

Experimental Results:

EDA:



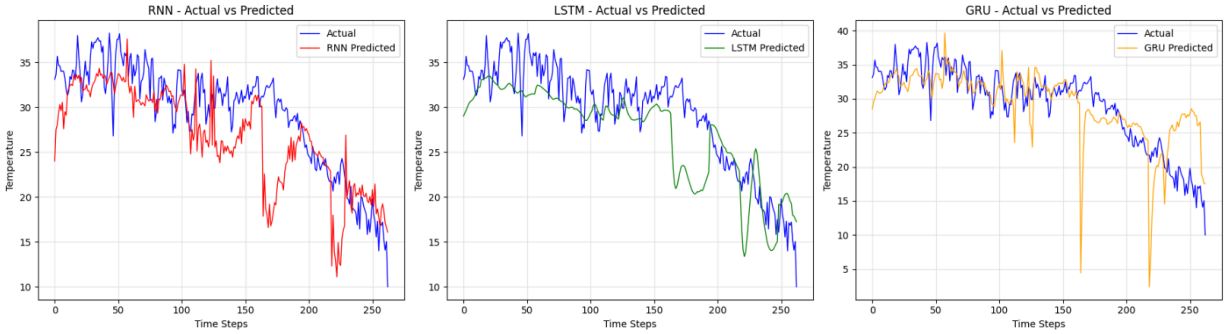


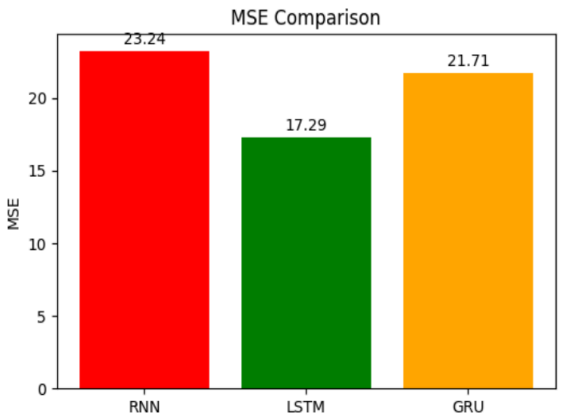


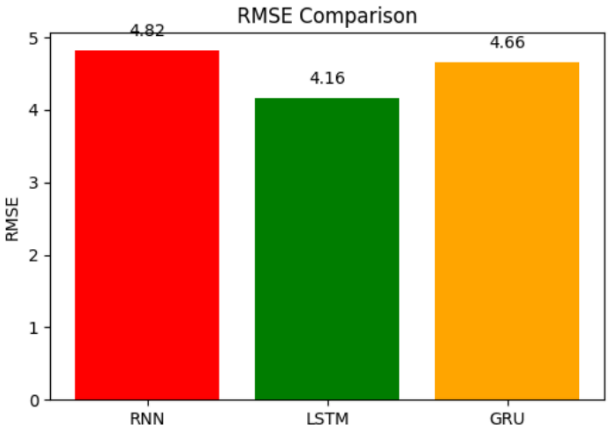
Metrics Analysis:

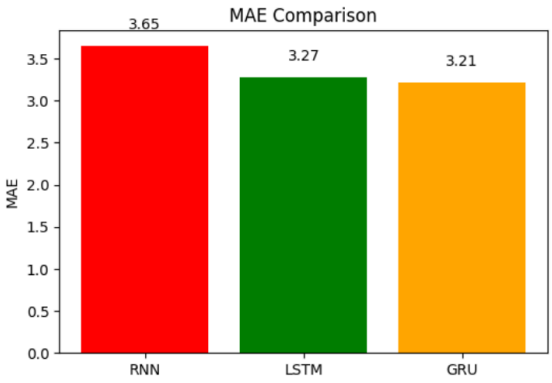
Model	MSE	RMSE	MAE	Execution Time (s)
RNN	23.2362	4.8204	3.6522	46.52
LSTM	**17.2911**	**4.1583**	**3.2726**	105.09
GRU	21.7120	4.6596	3.2129	106.85

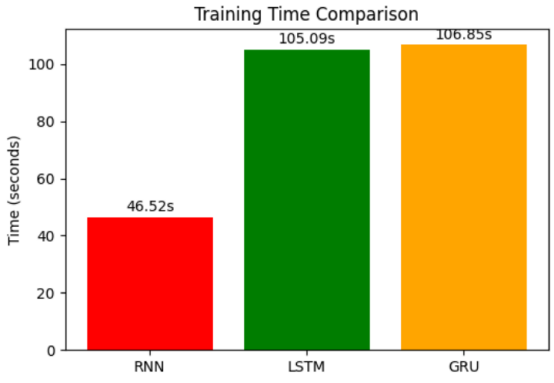
Comparison Graph of Actual vs Predicted of RNN, LSTM, GRU:











Result:

Thus, the comparative analysis of RNN, LSTM, and GRU has been successfully completed for multivariate climate time-series forecasting. The results show that **LSTM performs better overall** and provides more effective prediction performance for the given dataset.

The code file is available in my kaggle account:
https://www.kaggle.com/code/tamilin/comparison-rnn-lstm-gru-temperature-prediction

The code is available in my github account also:
https://github.com/tamil-azhagan/Deep-Learning-Exercises/blob/main/Projects/Ex7%20Comparison%20of%20RNN%2C%20LSTM%2C%20GRU/comparison-rnn-lstm-gru-temperature-prediction.ipynb

And the dataset is available at:
https://www.kaggle.com/datasets/sumanthvrao/daily-climate-time-series-data